REST API for Videogame Reviews: Technical Report

Hoe Yin Foong

Student ID: 201754233

GitHub Repository: https://github.com/hoeyinf/COMP3011-Coursework1

API Documentation: https://github.com/hoeyinf/COMP3011-Coursework1/blob/main/APIDocumentation.pdf

1. Planning

1.1 Database

The dataset used was Metacritic's Best Games and Reviews – 2025, containing 13000+ games and over a million reviews (Bayik, 2025). It consists of two csv files: one for games and one for reviews. This dataset was chosen as it contained easy-to-understand fields and recent data from up to March 2025. An Entity-Relationship Diagram (Appendix A) was designed and implemented using Django models (located in games/models.py).

A decision was made to split off Genre, Platform, Publisher, and Developer models from Game to make filtering Game more efficient.

1.2 API Endpoint Design

Full API endpoint description is in the API documentation (link on the cover page). API endpoints were designed while adhering REST principles, and the following structures were implemented:

- Collections resources use plurals and end with a forward slash e.g. **GET** `/games/`
- Single resources are identified using their id and do not end with a forward slash e.g. **GET** `/games/1`
- Actual endpoint implementations are prepended with `api` to make it clear they are API endpoints e.g. **GET** `/games/` is called using **GET** `/api/games/`

Endpoints are split between `/games/`, `/users/`, and `/reviews/`; these correspond with the Game, User, and Review models respectively. Two additional cases for retrieving a user or game's reviews use the sub-collections **GET** `/users/{user_id}/reviews/` and **GET** `/games/{game_id}/reviews/` respectively.

Full CRUD (**POST, GET, PATCH, DELETE**) was implemented only for reviews. Full user CRUD was considered but not implemented to keep the final solution focused on reviews.

Finally, using djangorestframework_jwt (explained in section 3), some **POST** `/token/` endpoints were created for user authentication.

2. Development

2.1 Python

Python was chosen for faster and easier development. There is good availability of widely used modern frameworks, such as FastAPI, Django, and Flask.

Past personal experience with python was another benefit. Python was used in previous modules, such as in COMP2011 Web Application Development. While an API service is not the same as a web application, there were many transferable skills that helped with project development, such as database management and project structuring.

The main drawback of python could be worse performance. However, considering that the project is relatively lightweight, the performance trade-off would not be significant.

2.2 Django

Django contains tools for full-stack development of a REST API:

- Object-relational mapping with its models, which is important for full CRUD.
- Simple-to-implement admin dashboard that makes database management much easier.
- Strong built-in authentication and security standards: built-in CSRF protection, and password encryption with a PBKDF2 algorithm and a SHA256 hash (Django Software Foundation, 2026).
- Ease of deployment to PythonAnywhere.

Another factor was that Django was covered in lecture material, which meant that less time spent learning it independently. In theory this would mean more time allocated to developing better APIs.

2.3 Django REST Framework (DRF)

DRF was another reason why Django was chosen. It is a toolkit that extends Django with features specifically for REST API development:

- Web-browsable API.
- Serializers for JSON serialization and deserialization.
- Pagination of responses (DRF's specific implementation, not Django's).
- Multiple authentication frameworks available (chosen method explained in section 3).
- The official DRF website also has extensive documentation and a tutorial for easier learning (Encode OSS LTD, 2026).

2.3.1 Web-browsable API

The ability to view the JSON responses in-browser makes it convenient to verify the correctness of returned data outside of code or the command line:

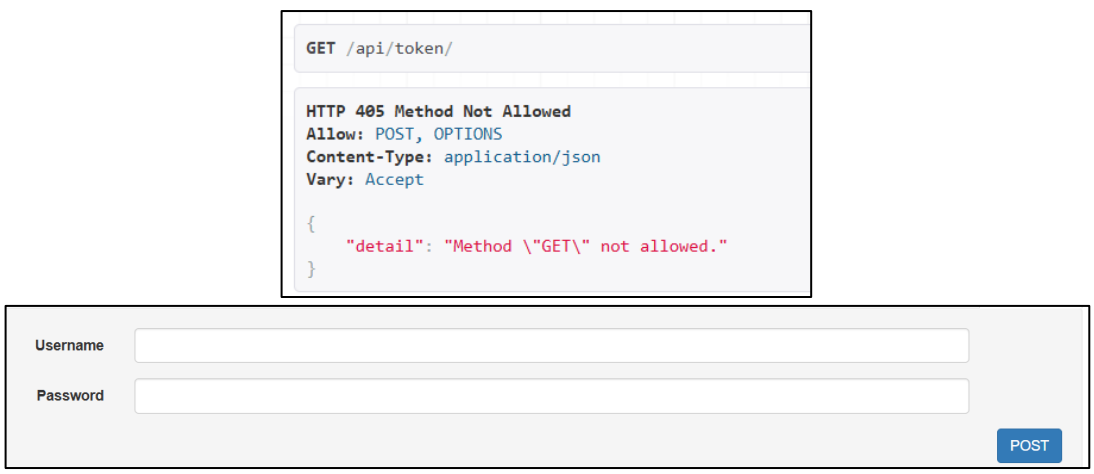


Figure 1: Browsable view for **GET** `/token/`

Figure 1 shows the appropriate response for **GET** `/token/` and provides a method to enter in data for **POST** `/token/`. Data can be entered in-browser which then returns:

```
POST /api/token/

HTTP 200 OK
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "refresh": "eyJhbGciOiJIUzI1NiIsInR5cC
    "access": "eyJhbGciOiJIUzI1NiIsInR5cCI
}
```

Figure 2: Response to valid **POST** `/token/`

2.3.2 Serializers

Serializers (defined in games/serializers.py) allow for the convenient serialization of JSON data.

DRF's `HyperLinkedModelSerializer` class uses hyperlinks to represent relationships. This class was subclassed into a `DynamicFieldsSerializer` (YAtOff, 2014), which overrides the initializer so that only the specified fields to serialize are returned (Appendix B). Most implemented serializers use this class to easily provide hyperlinks in their JSON responses, while limiting data to relevant fields.

An example usage of the `DynamicFieldsSerializer` is in Figure 3 below. Both the user and game serializers are subclasses of it; the `fields` parameter to specifies that the JSON representation of a review should return only those specified fields from the user and game serializers.

```
class ReviewSerializer(DynamicFieldsSerializer):
    """Serializer for a review."""
    user = UserSerializer(fields=['username', 'url'], read_only=True)
    game = GameSerializer(fields=['title', 'url'], read_only=True)
```

Figure 3: Nested Serialization of the user and game in ReviewSerializer

This also demonstrates nested serialization, another DRF feature that allows the embedding of related entities during serialization; serializers can use other serializers to create nested responses, shown in Figure 4:

```
GET /api/reviews/2

HTTP 200 OK
Allow: GET, POST, PATCH, DELETE, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

{
    "id": 2,
    "url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/2",
    "user": {
        "username": "MetalLiquido",
        "url": "https://hoeyinfoong.pythonanywhere.com/api/users/3"
    },
    "game": {
        "title": "The Legend of Zelda: Ocarina of Time",
        "url": "https://hoeyinfoong.pythonanywhere.com/api/games/1"
    },
    "date": "2022-05-04",
    "score": 100,
    "content": "Marcou minha infância, e realmente é um dos melhores jogos de todos os tempos!"
}
```

Figure 4: JSON response of **GET** `/reviews/2`

The main benefit is the improved discoverability of related resources via direct links. It also allows for easy customisation of response data; it's more useful than using just a string representation and does not overload the response with all the data from related entities.

2.3.3 Pagination

Certain endpoints can result in very long responses, such as **GET** `/games/` which can return all (10000+) games from the database. DRF has a `PageNumberPagination` class that can be used to easily split the response over multiple pages. This makes sure that the response size is manageable but still returns all the data requested in a navigable format. In-browser views can also load much faster. Details for which endpoints use pagination are in the API documentation.

3. JSON Web Token (JWT)

Authentication was implemented using the djangorestframework_jwt package. Users authenticate with their username and password at **POST** `/token/`, which will give them access and refresh JWTs. Details are in the API documentation.

Since authentication relies on using these temporary tokens, it reduces database load as users do not have to validate with their usernames and passwords every time. Alongside this improved scalability, authentication via JWT is stateless as the user state is never stored, keeping in-line with REST philosophy.

4. Testing

There was an attempt at test-driven development: tests were designed and implemented before implementing the actual API endpoint they were testing. It helped with ensuring the final implementation was more robust, and that further implementation of APIs would not break existing functionality.

Unit testing was done with the pytest, pytest-django, and requests packages. Tests are in the tests/ directory and split into three files: one each to test users, games, and reviews. In total, 76 unit tests were made to test all the endpoints; a list of tests is given in Appendix C.

Minor integration testing was "built into" the unit tests. For example, testing the **DELETE** `/reviews/{review_id}` endpoint initially uses the **POST** `/reviews/` endpoint to generate and retrieve a new review to delete. **POST** `/reviews/` itself uses **POST** `/token/` for authentication. This was due to shortsightedness of using the production database for testing, so some tests are not actual independent unit tests; however, the benefit is better test coverage using multiple endpoints together to verify correct behaviour. Appendix C clarifies which tests fit this description.

5. Deployment

PythonAnywhere was chosen for deployment as it is free, simple, and also supplies a HTTPS certificate by default with its non-custom domains (PythonAnywhere LLP, 2026b).

SSL/HTTPS was setup in live deployment in api_project/settings.py, and the code for that is commented out in the repository. HTTPS was forced using PythonAnywhere's Web settings (PythonAnywhere LLP, 2026a). This ensures a secure client-server communication.

6. Evaluation

6.1 Challenges

One unexpected challenge was the fact that PythonAnywhere has a file upload limit of 100MB; the resulting database could only use roughly 10% (130000) of reviews from the dataset. Most games in the database unfortunately do not actually have reviews. File upload limits will be taken into consideration for future projects.

6.2 Limitations and Future Development

As full CRUD was only implemented for reviews, an obvious extension is that CRUD could be added for users and games. The database can also be populated with the missing dataset data (90% of reviews) that could not be uploaded due to file limits. More complex analysis of user and game data could be done to provide more insightful trends and correlation with review scores.

7. Generative AI Declaration

7.1 Declaration and Usage

Microsoft 365 Copilot was used throughout planning and project development. Appendix D lists examples of prompts and responses.

During planning, AI was used to fully explore different possible frameworks for creating APIs (for all languages and frameworks other than Python and Django). It was asked to compare the benefits of different frameworks, such as Node.js, FastAPI, and DRF.

It was also asked to search for datasets pertaining to videogames. It was then used to parse the selected dataset to find missing values, and possible relationships that can be derived from the data. Finally, it was told to generate a python script to populate a Django model with the data, though this script had to be altered significantly.

During development, it was also used to explain how to implement features in DRF, such as pagination. It also helped to explain the different standard authentication methods available, to help choose between them.

7.2 Analysis

AI was most helpful when asking it to explain certain frameworks or features. Official documentation can be verbose, and AI provided concise clarifications on implementations of these features.

It was least helpful when it came to searching for datasets. Even after refining the prompt and explicitly asking for free and public datasets from Kaggle or Google Dataset Search, it would keep recommending datasets that did not contain the data needed (videogame reviews). A manual search had to be done.

As the project became more technically focused with more complex implementations of APIs, AI usage decreased as it ended up not giving accurate information; there was a shift to using official documentation and online forums (e.g. Stack Overflow) to solve specific needs.

8. References

Bayik, D. 2025. Metacritic's Best Games and Reviews - 2025. *Kaggle*. [Online]. [Accessed 26 February 2026]. Available from: https://www.kaggle.com/datasets/davutb/metacritic-games

Django Software Foundation. 2026. *Password management in Django*. [Online]. [Accessed 5 March 2026]. Available from: https://docs.djangoproject.com/en/6.0/topics/auth/passwords/

Encode OSS LTD. 2026. *Django REST Framework*. [Online]. [Accessed 5 March 2026]. Available from: https://www.django-rest-framework.org/

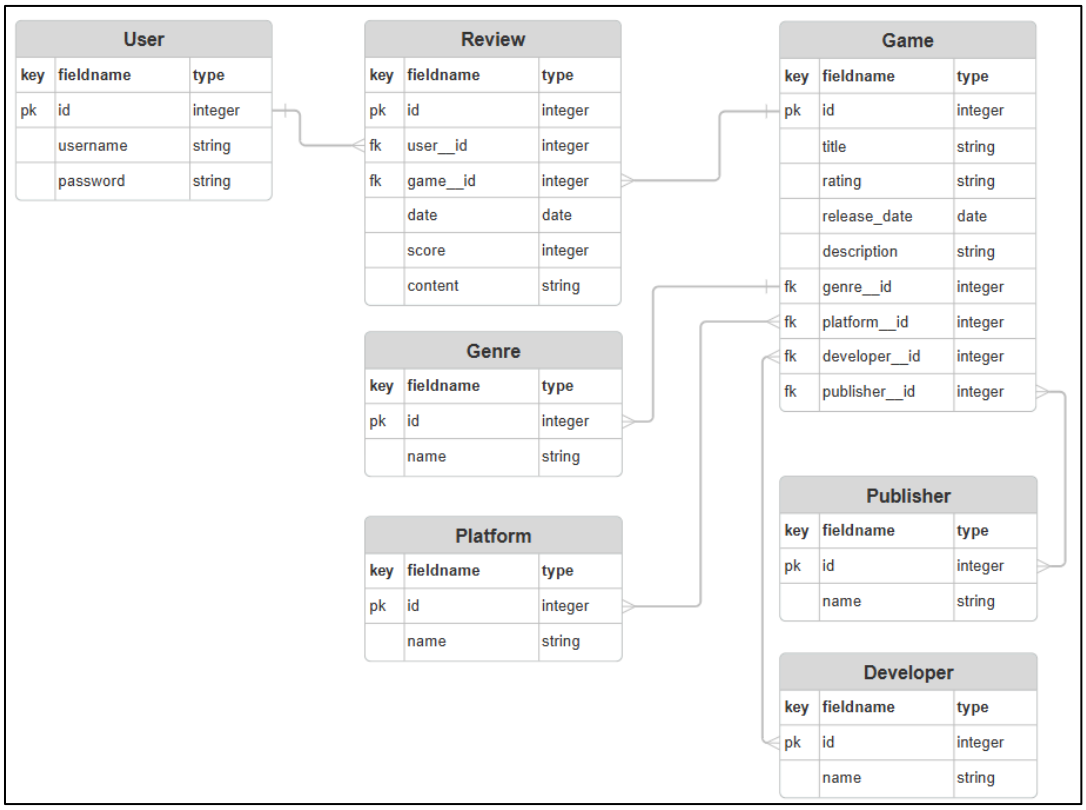
PythonAnywhere LLP. 2026a. *Forcing HTTPS*. [Accessed 19 Match 2026]. Available from: https://help.pythonanywhere.com/pages/ForcingHTTPS/

PythonAnywhere LLP. 2026b. *How to set up an HTTPS/SSL certificate for a custom domain*. [Accessed 19 Match 2026]. Available from: https://help.pythonanywhere.com/pages/HTTPSSetup/

YAtOff. 2014. You can override the serializer. 15 May. *Django Rest Framework: Dynamically return subset of fields*. [Online]. [Accessed 10 March 2026]. Available from: https://stackoverflow.com/questions/23643204/django-rest-framework-dynamically-return-subset-of-fields

9. Appendix

9.1 Appendix A – Entity-Relationship Diagram



9.2 Appendix B – DynamicFieldsSerializer's initializer

```python

class DynamicFieldsSerializer(s.HyperlinkedModelSerializer):
    """
    HyperlinkedModelSerializer that uses a fields argument to control which
    fields to display.
    """
    def __init__(self, *args, **kwargs):
        fields = kwargs.pop('fields', None)

        super().__init__(*args, **kwargs)

        # Keeps only fields that were specified
        if fields is not None:
            allowed = set(fields)
            existing = set(self.fields)
            for field_name in existing - allowed:
                self.fields.pop(field_name)
```

9.3 Appendix C – Unit tests for Videogame Review API

Note: In the **Name** column, some tests will have an asterik (*) followed by one or more API endpoints. This indicates that this test relies on the listed API endpoint(s) to function correctly. This

means they are technically not independent unit tests, but more accurately integration tests using multiple API endpoints together.

Those tests are written such that they will warn if the dependent functionality is not working properly, instead of the error being with the endpoint being tested.

Location in games/tests	API Endpoint Being Tested	Name	Description
test_api_users.py	**POST** `/token/`	`test_get_jwt()`	Uses valid, invalid, and missing credentials to verify that access and refresh tokens are being returned in the right scenarios. Also checks the HTTP statuses.
	POST `/token/refresh/`	`test_refresh_jwt()*` ***POST** `/token/`	Uses a valid, invalid, and missing refresh token to verify that an access token is being returned only when refresh is valid. Also checks the HTTP statuses.
	GET `/users/{user_id}`	`test_get()`	Uses a valid, invalid, and missing user_id to verify the correct data and HTTP statuses are being returned.
	GET `/users/{user_id}/reviews/`	`test_get_reviews()`	Uses a valid, invalid, and missing user_id to verify the correct review data and HTTP statuses are being returned.
		`test_get_reviews_size()`	Uses a valid user_id to check that the correct number of reviews is being returned.
	POST `/users/`	`test_post()`	Uses a variety of valid and invalid usernames and passwords to verify user creation. Also checks the HTTP statuses.
		`test_post_missing_credentials()`	Verifies that missing credentials return the correct HTTP status.
		`test_post_bad_syntax()`	Verifies that malformed query parameters return the correct HTTP status.
test_api_games.py	**GET** `/games/`	`test_get()`	Verfies that it returns a HTTP 200 OK and that it contains analytics.
		`test_category()`	Uses a variety of valid and invalid query parameters to

			verify that the correct HTTP statuses are being returned.
	GET `/games/{game_id}`	`test_get_id()`	Uses a valid, invalid, and missing game_id to verify the correct data and HTTP statuses are being returned.
	GET `/games/{game_id}/analytics`	`test_get_id_analytics()`	Uses a valid, invalid, and missing game_id to verify the correct HTTP statuses are being returned. Valid game_id should also return an average score and game data.
	GET `/games/{game_id}/reviews/`	`test_get_reviews()`	Uses a valid, invalid, and missing game_id to verify the correct HTTP statuses are being returned. Valid game_id should also have the correct review returned in the results.
	POST `/games/`	`test_method_not_allowed()`	Checks that a HTTP 405 Method Not Allowed is returned.
test_api_reviews.py	**GET** `/reviews/{review_id}`	`test_get_id()`	Uses a valid, invalid, and missing review_id to verify the correct HTTP statuses are being returned. Valid review_id should also return the correct data.
	POST `/reviews/`	`test_post_invalid_request()*` *__POST__ `/token/`	Checks that missing credentials, existing reviews, and invalid or malformed query parameters return the correct error message and HTTP statuses.
		`test_post_invalid_values()*` *__POST__ `/token/`	Uses a variety of bad values for the date and score of the review to check that the correct HTTP responses are returned and a review is not created.
		`test_post_valid()*` *__POST__ `/token/`, **DELETE** `/reviews/{review_id}`	Uses good data and credentials to verify review creation and HTTP status. Also deletes the new review afterwards (or reminds tester

			to do it manually if deletion fails).
	PATCH `/reviews/{review_id}`	`test_patch_invalid_request()*` *__POST__ `/token/`	Checks that invalid or missing review_id, missing credentials, wrong credentials, and invalid or malformed query parameters return the correct error message and HTTP statuses.
		`test_patch_invalid_scores()*` *__POST__ `/token/`	Tests that bad (boundary and invalid) values for the score query parameter. Checks that a HTTP 400 Bad Request is returned.
		`test_patch_valid()*` *__POST__ `/token/`	Uses a variety of valid query parameters to check that the correct data is being updated. Also checks HTTP statuses. If successful, uses the endpoint again to reset the review to its original state.
	DELETE `/reviews/{review_id}`	`test_delete()*` *__POST__ `/token/`, **POST** `/reviews/`, **GET** `/users/{user_id}/reviews/`	Checks that invalid or missing review_id, missing credentials, and wrong credentials return the correct HTTP status. If it fails, it will use try to delete the corresponding review from the user (or tell the tester to do it manually)

9.4 Appendix D – Generative AI Example Prompts and Responses

Please note that full responses are not shown.

9.4.1 Finding frameworks for REST API development

What are possible frameworks for REST API development

Copilot

Here's a clear, well-organized list of **popular and widely used frameworks for REST API development** in 2026, backed by recent sources.

✅ Popular Frameworks for REST API Development (2026)

1. Express.js (JavaScript / Node.js)

Express.js remains one of the **most widely used** frameworks for building REST APIs because it is:

- Simple and flexible
- Easy to set up with minimal boilerplate
- Ideal for small–medium APIs and startups

analyticsinsight

---

2. Django REST Framework (Python)

Django REST Framework (DRF) is favored for:

- Strong built-in security and authentication
- Mature ecosystem and reliability

9.4.2 Comparison of REST API frameworks

Compare these frameworks, listing their strengths and weaknesses

Below is a **clean, citation-backed comparison** of the major REST API frameworks you asked about, with their **strengths and weaknesses clearly listed**.

✅ Framework Comparison: Strengths & Weaknesses

The following information is drawn from recent 2025–2026 analyses of API frameworks.

---

1. Express.js (JavaScript / Node.js)

Source: Analytics Insight (2026) anakin

✅ Strengths

- **Simple and flexible**, allowing developers to set up APIs with minimal boilerplate. anakin
- **Fast to prototype**, ideal for small and medium projects.
- **Huge ecosystem** via NPM, providing thousands of middleware options.

❌ Weaknesses

- **Not opinionated**, meaning developers must create their own structure, leading to inconsistency in large teams.
- **Less suitable for large-scale enterprise systems** where stronger architecture is needed.

---

2. Django REST Framework (Python)

Source: Analytics Insight (2026) anakin

✅ Strengths

- **Excellent built-in security**, permissions, and authentication tools. anakin
- **Mature ecosystem** with stable, well-tested components.
- **Great for large business applications**, dashboards, and data-rich systems.

❌ Weaknesses

- **Slower performance** compared to async-first frameworks like FastAPI.
- **Heavier and more opinionated**, which can be overkill for small microservices.

9.4.3 Finding datasets on videogame reviews

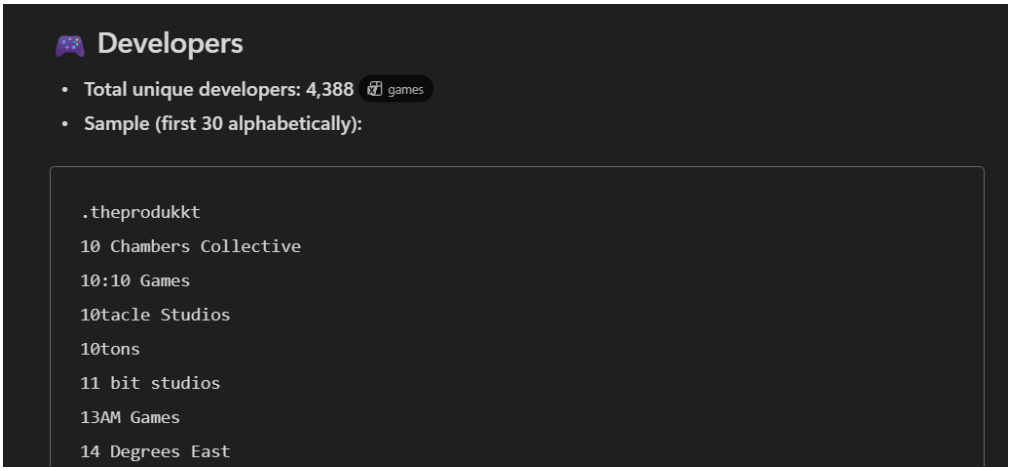
Note that none of these datasets were chosen. The first dataset was promising but based off randomly generated data (not reflective of real-life data).

Find public free datasets on videogames that contain user reviews or ratings, preferably from Kaggle or Google Dataset Search.

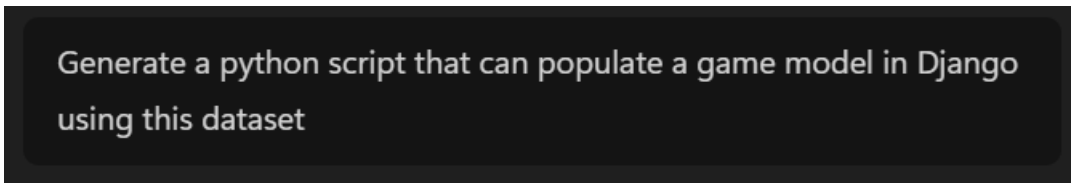
✅ Summary Table

Dataset	Contains User Ratings?	Contains User Reviews?	Free?
Kaggle – Video Game Reviews & Ratings	✅ Yes	✅ Some	✅
Kaggle – Steam Reviews (1M reviews)	✅ Yes	✅ Yes (text)	✅
GitHub Game-Datasets (Metacritic, OpenCritic, etc.)	✅ Yes	❓ Some, depends on API	✅
Google Dataset Search (discovery)	✅ Yes	✅ Often	✅
GTS Top Games Dataset	✅ Yes	❓ Not full text	✅

9.4.4 Analysing a dataset



9.4.5 Populating a databse



2) Management command to load the CSV

Create `app/management/commands/load_games.py`:

```python
# app/management/commands/load_games.py
import csv
import math
from datetime import datetime
from pathlib import Path
from typing import Iterable, List, Optional, Tuple

from django.core.management.base import BaseCommand, CommandError
from django.db import transaction
from django.db.models import Q

from app.models import (
    Game, Genre, Platform, Developer, Publisher
)
```

9.4.6 Exploring authentication methods

Explain how authentication can be done in a REST API.

🔐 3. JWT (JSON Web Tokens)

JWTs are self-contained tokens that include user data (claims) and are cryptographically signed.

Typical JWT flow

1. User logs in.
2. Server issues a JWT signed with secret/private key.
3. Client sends JWT with each request.
4. Server validates signature but does **not** need to store session data.

Pros

- Stateless (ideal for scaling).
- Widely supported.

Cons

- Cannot be invalidated easily before expiration.
- Must be stored securely on client side.

🔐 4. OAuth 2.0

An industry-standard authentication and authorization framework—commonly used by Google, Microsoft, GitHub, etc.

9.4.7 Explaining authentication methods

Explain JWT, OAuth 2.0, and API keys in more detail.

🔐 1. JSON Web Tokens (JWT)

✅ What is a JWT?

A **JWT (JSON Web Token)** is a compact, URL-safe, cryptographically signed token used to securely transmit claims between parties. It is commonly used for stateless authentication in REST APIs.

A JWT consists of **three Base64URL-encoded parts**:

1. **Header** – describes the token type and signing algorithm
2. **Payload** – contains the claims (user data)
3. **Signature** – verifies token integrity

✅ Structure Example

```
xxxxx.yyyyy.zzzzz
```

✅ How JWT Authentication Works

1. User logs in with credentials.
2. Server verifies and issues a JWT signed with a secret/private key.
3. Client stores JWT (usually in memory or local storage).
4. Client includes the JWT in the `Authorization: Bearer` header of each request.
5. Server verifies the token signature and extracts claims—without hitting a database (stateless).

✅ Pros

- Scalable (no server-side session storage)
- Fast validation via signature
- Can encode roles, permissions, and metadata
- Works well across microservices

❌ Cons

- Hard to revoke before expiration (stateless tokens)
- Must be stored safely on client side
- Large payloads increase request sizes